Market-Basket Analysis on the IMDB Top 1000 Dataset

Your Name — Student ID: 73427A
Master in Data Science for Economics
Algorithms for Massive Data — A.Y. 2025/26

June 2026

Declaration

I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work, and including any code produced using generative AI systems. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study.

1 Introduction

Market-basket analysis is a classic data mining technique that identifies sets of items frequently occurring together in a collection of transactions. Originally developed to analyze retail purchases, where each "basket" is a customer transaction and the "items" are the products bought, the same framework applies to any domain where we want to discover items that co-occur more often than a given threshold.

In this project we apply market-basket analysis to movie data. We treat each movie as a basket and the actors starring in it as the items, in order to discover groups of actors who frequently appear together. We implement and compare two algorithms studied in the course: A-Priori and PCY (Park–Chen–Yu). The main goals are to find the frequent itemsets of actors, to verify that both algorithms produce the same result, and to study how they scale with the size of the data.

2 Dataset

We use the *IMDB Movies Dataset* published on Kaggle, released under the Public Domain license. The dataset contains the 1,000 top-rated movies and TV shows, with information such as title, release year, genre, rating, director and the four leading actors of each title.

For the purpose of this project we consider only the four columns describing the leading actors: `Star1`, `Star2`, `Star3` and `Star4`. Each movie therefore contributes one basket containing up to four actors. The dataset is downloaded programmatically through the Kaggle API, so that the whole analysis is reproducible from the notebook.

A first inspection shows that the dataset contains 1,000 movies and 2,709 distinct actors across the four star columns, with no missing values in those columns. Since each movie lists exactly four actors, the baskets all have a fixed size of four items. This is an important feature of the dataset: compared to a retail setting, where baskets can contain dozens of items, here each basket is small and of constant size, which will later help explain the relative behaviour of the two algorithms.

3 Data organization and preprocessing

3.1 Building the baskets

The raw data is a table with one row per movie. From each row we extract the four actor columns and build a *basket*, represented as a Python `set` of actor names. Using a set has two advantages: it automatically removes any actor accidentally listed twice within the same movie, and it provides fast membership tests, which are useful when counting itemsets later.

Formally, let I be the set of all distinct actors appearing in the dataset. Each movie m is mapped to a basket $B_m \subseteq I$, and the whole dataset becomes a collection of baskets $\mathcal{B} = \{B_1, B_2, \ldots, B_n\}$, where n is the number of movies.

3.2 Cleaning the actor names

Although the star columns of this particular dataset have no missing values, we still apply a cleaning step so that the code behaves correctly on noisier data as well. For each actor value we:

- treat genuine missing values (`None` or `NaN`) as absent;
- strip leading and trailing whitespace;
- discard empty strings and the literal string `"nan"`, which can appear when a missing value is converted to text.

After cleaning, baskets that end up empty are removed. On the current dataset this leaves all 1,000 baskets intact, but the same code would correctly handle a dataset with missing or malformed actor entries. This is consistent with the requirement that the techniques scale to larger and possibly messier datasets.

3.3 Subsampling

Following the project guidelines, a global variable controls whether the analysis runs on the full dataset or on a random subsample. When subsampling is enabled, a fixed random seed guarantees reproducible results. For the experiments reported here we use the whole dataset, since 1,000 movies are easily handled in memory.

4 Algorithms

We implement two algorithms for finding frequent itemsets: A-Priori and PCY. Both take as input the collection of baskets and a *support threshold*, and return all itemsets whose support reaches that threshold.

4.1 Support and the monotonicity property

The *support* of an itemset is the number of baskets that contain it. We specify the threshold as a relative frequency s and convert it into an absolute count $s \cdot n$, where n is the number of baskets. An itemset is *frequent* if its support is at least $s \cdot n$. Expressing the threshold as a relative value makes the results comparable across datasets of different sizes, which is essential for the scalability experiments.

Both algorithms rely on the *monotonicity* (or downward-closure) property: if an itemset is frequent, then all of its subsets are also frequent. Equivalently, if any subset of an itemset is infrequent, the itemset itself cannot be frequent. This property lets us avoid generating most candidate itemsets, since we only need to consider candidates whose subsets are all known to be frequent.

4.2 A-Priori

A-Priori finds frequent itemsets level by level, increasing the itemset size by one at each pass over the data.

- **Pass 1.** Count the occurrences of every single item and keep those that are frequent.
- **Pass k.** Generate candidate itemsets of size k from the frequent itemsets of size $k-1$, count them in a scan over the baskets, and keep the frequent ones.

Candidate generation has two steps. In the *join* step, two frequent $(k-1)$-itemsets are combined into a candidate of size k only if they share their first $k-2$ items (when items are kept in a fixed order). In the *pruning* step, a candidate is discarded immediately if any of its $(k-1)$-subsets is not frequent, as allowed by the monotonicity property. This keeps the number of candidates that must actually be counted as small as possible.

In our implementation the baskets are scanned once per level. We stop at itemsets of size three: since each basket contains only four actors, itemsets of size four would be extremely rare, and larger ones are impossible. This choice is therefore dictated by the structure of the data rather than being an arbitrary limit.

4.3 PCY (Park–Chen–Yu)

PCY improves the second pass of A-Priori, which is usually the most memory-intensive, because the number of candidate pairs can be very large. The key observation is that during the first pass, while counting single items, the algorithm reads every basket anyway and can do extra work at almost no additional I/O cost.

During the first pass, in addition to counting single items, PCY hashes every pair of items in each basket into a fixed number of buckets and counts how many pairs fall into each bucket. The first pass of our implementation is shown below.

```
def pcy_first_pass(baskets, min_count, n_buckets):
    item_counts = Counter()
    bucket_counts = np.zeros(n_buckets, dtype=np.int64)

    for basket in baskets:
        items = sorted(basket)
        for item in items:
            item_counts[item] += 1
        for a, b in combinations(items, 2):
            bucket_counts[pair_hash(a, b, n_buckets)] += 1

    frequent_items = {item for item, c in item_counts.items() if c >= min_count}
    frequent_buckets = bucket_counts >= min_count
    return item_counts, frequent_items, frequent_buckets
```

After the first pass, a bucket is called *frequent* if its count reaches the support threshold. The crucial point is the following: if a pair is frequent, then every basket containing it also contributes to its bucket, so that bucket must be frequent as well. By contraposition, a pair that hashes to an *infrequent* bucket cannot be frequent and can be discarded before the second pass.

This gives PCY an extra filter for generating candidate pairs. In A-Priori, every pair of frequent items is a candidate; in PCY, a pair is a candidate only if both items are frequent *and* the pair hashes to a frequent bucket. The bucket counts are stored as a compact array of integers, and only a bitmap of frequent buckets is kept for the second pass, which is far smaller than storing a count for every candidate pair.

From the third pass onwards, PCY behaves exactly like A-Priori, because the bucket trick only helps with pairs. In our implementation the two algorithms therefore share the same code for generating and counting itemsets of size three.

5 Scalability

A central requirement of the project is that the techniques used must scale to larger datasets. We address scalability in two ways: by writing the algorithms so that their cost grows gracefully with the data, and by measuring empirically how their running time behaves as the number of baskets increases.

5.1 Design for scalability

Both algorithms scan the baskets a fixed number of times (one pass per itemset size) and never materialize the full set of possible itemsets. The monotonicity-based pruning keeps the number of candidates small, and PCY further reduces the candidate pairs through the bucket filter. The support threshold is always expressed as a relative frequency, so the same threshold is meaningful regardless of the dataset size. These choices ensure that the memory footprint stays bounded and that the work per basket does not grow with the total number of baskets.

5.2 Generating larger datasets by replication

The dataset provided for this project contains only 1,000 movies, which is not enough to observe how the algorithms behave on larger inputs. To study scalability we therefore generate larger synthetic datasets by *pure replication*: the original baskets are repeated until the desired number of baskets is reached.

Pure replication has a precise and useful property. If every basket is repeated the same number of times, the support of every itemset grows by the same factor as the number of baskets, so its *relative* support is preserved. As a consequence, the set of frequent itemsets is exactly the same at every dataset size: only their absolute counts change. This is intentional, because it lets us isolate the effect of data *volume* on the running time, keeping the structure of the frequent itemsets fixed. In other words, replication acts as a controlled experiment in which the only variable that changes is the number of baskets to be processed.

We generate datasets of 10,000, 50,000, 100,000 and 200,000 baskets, starting from the original 1,000. We deliberately keep the original 1,000-basket dataset as a reference but exclude it from the timing plots, since at that scale the running times are so small that they are dominated by measurement noise.

6 Experiments

6.1 Experimental setup

All experiments use a relative support threshold of $s = 0.003$, meaning that an itemset is frequent if it appears in at least three movies. This value was chosen after a small exploration of several thresholds, reported in Section 6.2. Running times are measured with Python's `perf_counter`, timing only the algorithm itself and excluding data loading and basket construction. Each measurement is repeated seven times and we keep the minimum, which is the run least affected by background activity and therefore the most stable estimate of the computation cost.

6.2 Choosing the support threshold

Because each basket contains only four actors and most actors appear in very few movies, the number of frequent itemsets is very sensitive to the threshold. Table 1 shows how the number of frequent itemsets of each size changes with the threshold.

We selected $s = 0.003$ as a compromise: it yields enough frequent pairs and triples to make the analysis meaningful, while keeping the criterion selective enough to be significant (an itemset must appear in at least three movies).

Support	Min. movies	Size 1	Size 2	Size 3
0.005	5	79	3	1
0.004	4	145	5	1
0.003	3	271	25	3
0.002	2	641	121	25

Table 1: Number of frequent itemsets for different support thresholds.

6.3 Correctness

As a sanity check, we verified that A-Priori and PCY return exactly the same frequent itemsets on the original dataset: both find 299 frequent itemsets in total, and the two results are identical. This confirms that the PCY optimization changes only the efficiency of the computation, not its result.

6.4 Effectiveness of the PCY bucket filter

To understand the behaviour of PCY, we measured how many candidate pairs survive its bucket filter. With 79 frequent single actors, A-Priori would generate $\binom{79}{2} = 3{,}081$ candidate pairs. After applying the bucket filter, PCY retains only 7 candidate pairs: the filter removes 99.8% of them. The filter is therefore extremely effective in relative terms. As discussed in Section 7, however, this does not translate into a faster algorithm, because the absolute number of pairs involved is already very small.

6.5 Running time and scalability

Table 2 reports the running time of both algorithms at each dataset size, together with the speed-up of PCY relative to A-Priori (the ratio of the two times). Figure 1 plots the running times for the levels of 10,000 baskets and above.

Baskets	A-Priori (s)	PCY (s)	Speed-up
1,000	0.0134	0.0132	1.009
10,000	0.0540	0.1051	0.514
50,000	0.2433	0.5379	0.452
100,000	0.4810	1.0535	0.457
200,000	0.9915	2.1796	0.455

Table 2: Running time of A-Priori and PCY at different dataset sizes. The speed-up is the ratio of A-Priori's time to PCY's time; values below 1 mean PCY is slower.

Two observations stand out. First, the running time of both algorithms grows linearly with the number of baskets: when the data doubles, the time roughly doubles. This confirms that the approach scales with data volume. Second, PCY is consistently slower than A-Priori on this dataset, by an almost constant factor of about 2.2, regardless of the dataset size. The interpretation of this result is discussed in the next section.

7 Discussion

7.1 Interpreting the frequent itemsets

The frequent itemsets found are not random co-occurrences: they correspond to recognizable patterns in cinema. Several frequent pairs and triples come from film *sagas* whose cast is stable

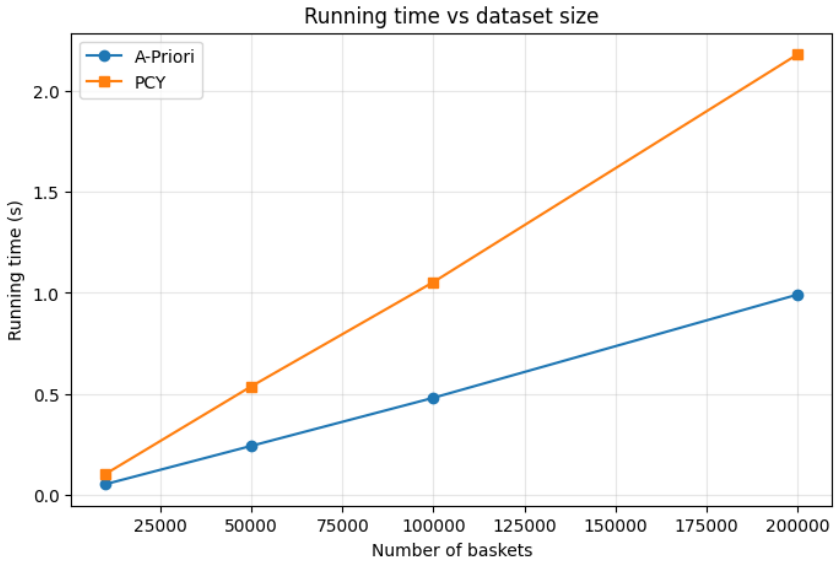

Figure 1: Running time of A-Priori and PCY as a function of the number of baskets.

across instalments, such as the lead trio of the Harry Potter series (Daniel Radcliffe, Emma Watson and Rupert Grint), the fellowship of The Lord of the Rings (Elijah Wood, Ian McKellen and Orlando Bloom) and the original Star Wars cast (Mark Hamill, Harrison Ford and Carrie Fisher). Others reflect recurring *director–actor collaborations*, such as Toshirô Mifune in the films of Akira Kurosawa or John Turturro in those of the Coen brothers. A further group comes from shared cinematic universes, most notably several actors of the Marvel films (for example Robert Downey Jr., Chris Evans and Scarlett Johansson). This shows that the algorithm recovers meaningful structure from the data, and not merely statistical noise.

7.2 Why A-Priori outperforms PCY here

The most interesting result of the project is that PCY, despite being an optimization of A-Priori, is consistently slower on this dataset. At first this seems to contradict the theory, but it follows directly from it once the assumptions behind PCY are made explicit.

PCY is designed for the case in which the second pass of A-Priori is a genuine bottleneck: when there are so many candidate pairs that the table of their counts does not fit in memory. Its bucket filter pays off precisely when it removes a large *absolute* number of pairs that would otherwise have to be counted. This situation arises with dense baskets containing many items, where the number of candidate pairs is huge.

Our dataset is the opposite case. Each basket contains only four actors, so it produces at most six pairs, and the number of frequent actors is small. Even though the bucket filter is extremely effective in relative terms, removing 99.8% of the candidate pairs, the absolute number of pairs involved is tiny: A-Priori only has to count a few thousand pairs, which it does almost instantly. The saving from the filter is therefore negligible in terms of time.

At the same time, PCY pays a real cost that A-Priori does not: during the first pass it must hash every pair of every basket. Under pure replication this cost grows linearly with the number of baskets, reaching over a million hashing operations at 200,000 baskets. Since the benefit of the filter is negligible while the hashing cost grows with the data, PCY ends up about twice as slow as A-Priori, and the gap is a constant factor because both the cost and the data scale linearly.

In short, PCY does not underperform because it is implemented poorly, but because the dataset does not exhibit the conditions for which PCY was designed. This is itself a useful finding: the best algorithm depends on the structure of the data, not only on its theoretical

guarantees.

8 Conclusions

We implemented and compared two market-basket algorithms, A-Priori and PCY, on the IMDB Top 1000 dataset, treating each movie as a basket and its leading actors as items. Both algorithms find the same frequent itemsets, which correspond to meaningful patterns such as film sagas, director–actor collaborations and shared cinematic universes. Using replication to generate larger datasets, we showed that both algorithms scale linearly with the number of baskets. Finally, we found that A-Priori is faster than PCY on this data, and we explained why: with small, fixed-size baskets and few candidate pairs, the bucket filter of PCY brings a negligible absolute saving while adding a hashing cost that grows with the data. The experiment thus illustrates that the advantage of an algorithm depends on whether the data matches the assumptions it was designed for.